Slides for P3104R3
Bit permutations

Document number:
P3730R0

Date:
2025-06-04

Audience:
LEWG

Project:
ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Reply-to:
Jan Schultke <janschultke@gmail.com>

Source:
github.com/Eisenwave/cpp-proposals/blob/master/src/bit-permutations-slides.cow

---

> **IMPORTANT:** This document has custom controls:
>
> - →, ↓: go to the next slide
> - ←, ↑: go to previous slide

Bit permutations

P3104R3

History

- `<bit>` functions added by **P0553R4**: *Bit operations* (C++20)
  - simple utilities like `has_single_bit`
  - instruction wrappers like `rotl`, `popcount`, `countl_zero`, …
- `<stdbit.h>` functions added by **N3022**: *Modern Bit Utilities* (C23)
- P3104R0 continues work on `<bit>`
  - first seen in LEWGI at Tokyo 2024
  - removal of some functions requested to increase consensus
  - proposal minified twice since then; four function templates remain

Synopsis

```
template<unsigned-integer T>
  constexpr T bit_repeat(T x, int length) /* not noexcept */;

template<unsigned-integer T>
  constexpr T bit_reverse(T x) noexcept;

template<unsigned-integer T>
  constexpr T bit_compress(T x, T m) noexcept;

template<unsigned-integer T>
  constexpr T bit_expand(T x, T m) noexcept;
```

`unsigned-integer` means "*Constraints*: `T` is an unsigned integer type".

std::bit_repeat

```
template<unsigned-integer T>
  constexpr T bit_repeat(T x, int length) /* not noexcept */;
```

- repeats/broadcasts bit pattern with `length` in `x`
- hardware support depends on `length` (> 0)

```
bit_repeat(0b****'****'****'0110u, 4)
```

```
        == 0b0110'0110'0110'0110u
```

std::bit_reverse

```
template<unsigned-integer T>
  constexpr T bit_reverse(T x) noexcept;
```

- reverses the order of bits in x
- supported by `rbit` (ARM), `bswap` (x86), …

```
bit_reverse(0b1000'1111'0000'1010u)
                  ┌──────────────┐
                ┌─┼────────────┐ │
                │ │            │ │
                │ │     …      │ │
          == 0b0101'0000'1111'0001u
```

std::bit_compress

```
template<unsigned-integer T>
  constexpr T bit_compress(T x, T m) noexcept;
```

- packs bits of `x` where the "mask" `m` has a one-bit, contiguously
- supported by `bext` (ARM), `pext` (x86)

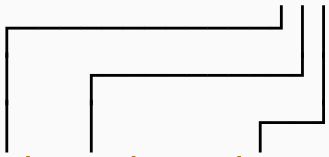
```
unsigned m = 0b0001'0010'0000'1000u;
bit_compress(0b***1'**1*'****'0***u, m)
                   |    |         |
                   |    |         |
                   |    |         |
          == 0b0000'0000'0000'0110u
```

std::bit_expand

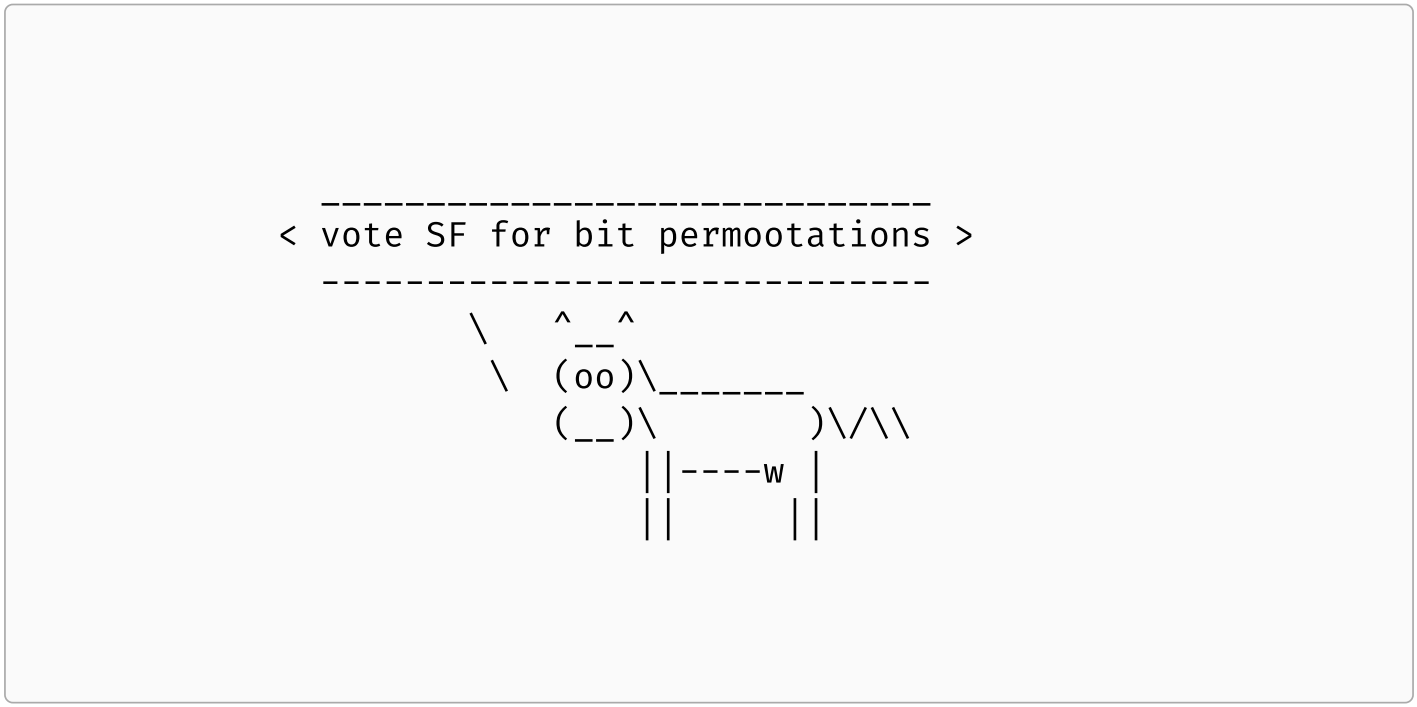
```
template<unsigned-integer T>
  constexpr T bit_expand(T x, T m) noexcept;
```

- opposite of `std::bit_compress`
- supported by `bdep` (ARM), `pdep` (x86)

```
unsigned m = 0b0001'0010'0000'1000u;
bit_expand(  0b****'****'****'*110u, m)
```



```
           == 0b0001'0010'0000'0000u
```


vote SF for bit permootations